

# **Laboratorio di Sistemi Operativi**

**Corso di Laurea in Informatica**

*A.A. 2025-2026*

Alberto Finzi

# Introduzione

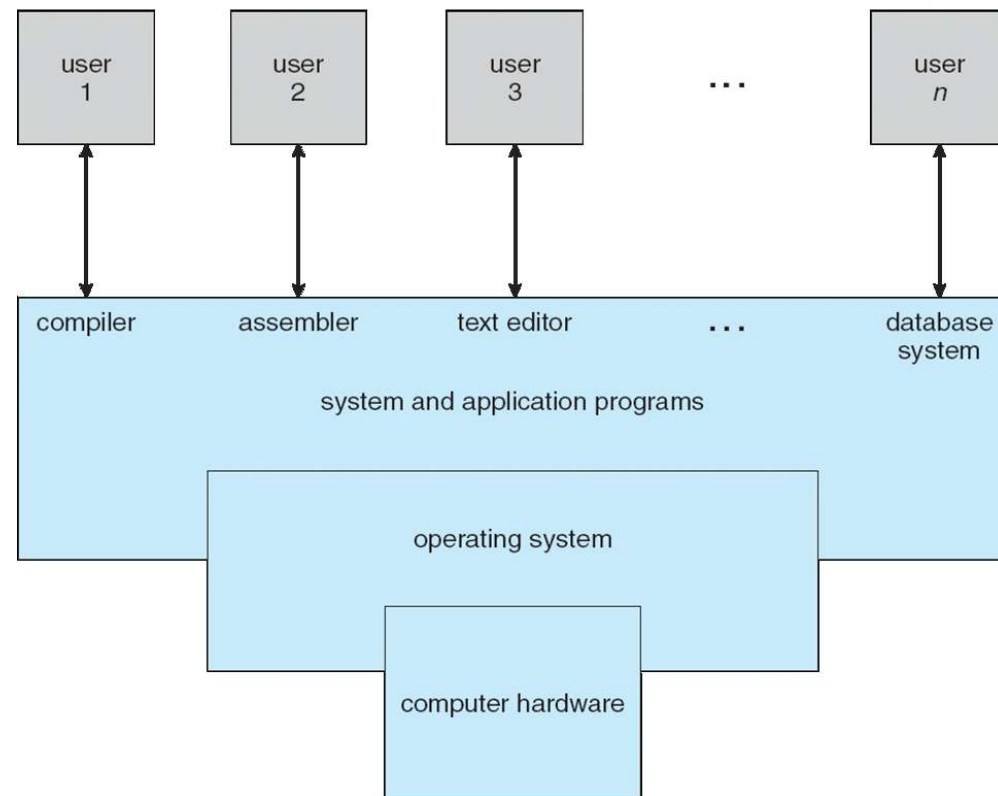
Il SO è un programma che gestisce le risorse di un calcolatore e fa da intermediario tra l'utente di un calcolatore e l'hardware del calcolatore

Obiettivi di un Sistema Operativo:

- Eseguire i programmi utente e semplificare l'interazione con il calcolatore
- Rendere efficace ed efficiente l'utilizzo del calcolatore
- Utilizzare l'hardware in modo efficiente

# Introduzione

Il SO è un programma che gestisce le risorse di un calcolatore e fa da intermediario tra l'utente di un calcolatore e l'hardware del calcolatore



# Introduzione

- L'SO è un **allocatore di risorse**
  - Gestisce tutte le risorse del calcolatore:
  - Decide tra richieste conflittuali per l'assegnazione corrette ed efficiente delle risorse
- L'SO è un **programma di controllo**
  - Controlla l'esecuzione dei programmi evitando errori e usi impropri del calcolatore
- L'SO come **macchina estesa**
  - Fornisce un'astrazione fornendo un ambiente coerente ed uniforme per l'esecuzione dei programmi

# Introduzione

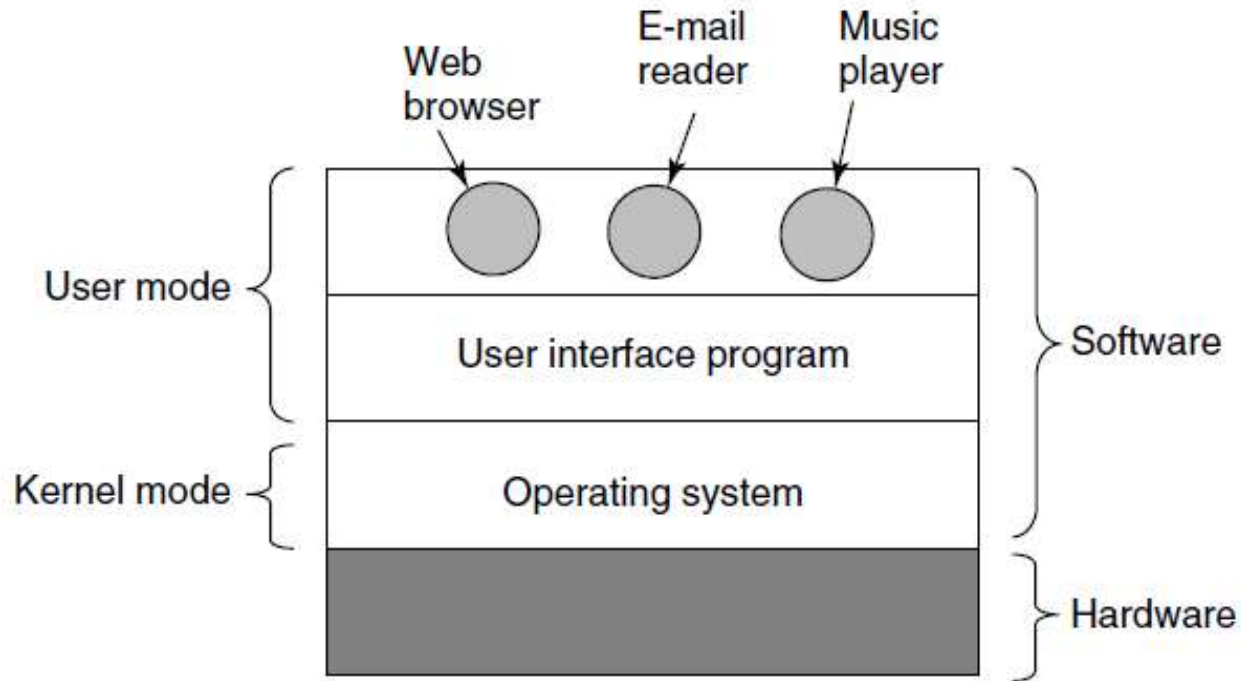
Il nucleo del sistema (**kernel**) gestisce le risorse essenziali: la CPU, la memoria, le periferiche, etc.

Tutto il resto, anche l'interazione con l'utente, è ottenuto tramite programmi eseguiti dal **kernel**, che accedono alle risorse hardware tramite delle richieste a quest'ultimo.

Il kernel è il solo programma ad essere eseguito in **modalità privilegiata**, con il completo accesso all'hardware, gli altri programmi vengono eseguiti in modalità protetta.

# Introduzione

Il nucleo del sistema (**kernel**) gestisce le risorse essenziali: la CPU, la memoria, le periferiche, etc.



# Introduzione

Il nucleo (**kernel**) gestisce le risorse essenziali: la CPU, la memoria, le periferiche, etc.

Tutto il resto:

se non è **programma di sistema** (utilità fornite con il SO) ...

... è un **programma applicativo**

Tutti i SO forniscono **servizi**. Eseguire un nuovo programma, aprire un file, allocare la memoria sono esempi di servizi forniti da un sistema operativo.

Descriveremo servizi forniti da varie versioni del sistema operativo Unix.

# Introduzione

Risorse gestite dal kernel

**Gestione dei  
processi**

**Gestione  
della  
memoria**

**Gestione dei  
file**

**Gestione  
della  
memoria di  
massa**

**Gestione  
della cache**

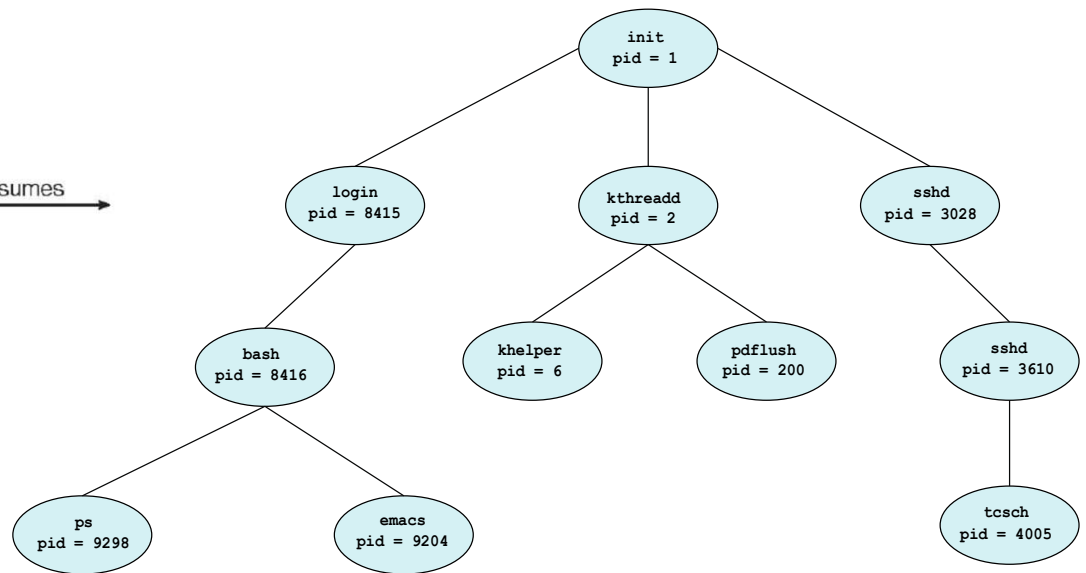
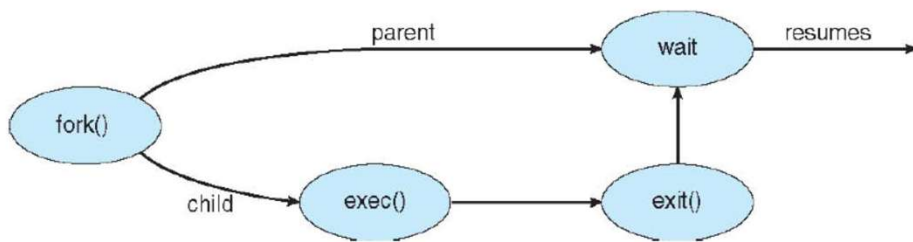
**Gestione  
dell'I/O**



# Introduzione

## Gestione dei processi

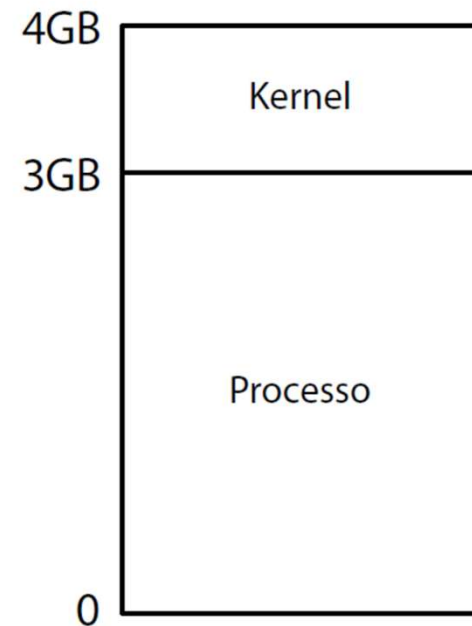
- Creazione e cancellazione di processi/thread
- Sospensione e ripresa di processi/thread
- Sincronizzazione di processi/thread
- Comunicazione tra processi/thread
- Gestione del deadlock



# Introduzione

## Gestione della memoria principale

- Per eseguire i programmi istruzioni e dati devono essere caricati in **memoria principale**
- Attività di gestione della memoria:
  - Gestire spazi di indirizzamento virtuale
  - Tracciare e proteggere memoria allocata
  - Decidere processi e dati da spostare dentro e fuori alla/dalla memoria
  - Allocazione e deallocazione dello spazio di memoria secondo necessità



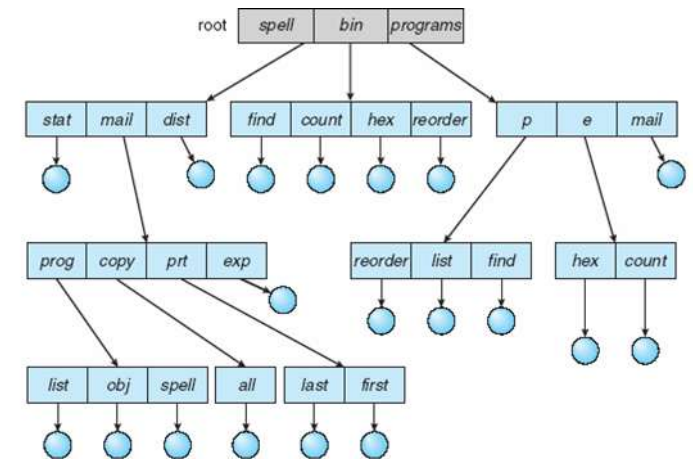
# Introduzione

## Gestione memoria di massa e file

- Il sistema operativo fornisce una rappresentazione logica e uniforme dell'archiviazione delle informazioni

## Gestione del **File-System**

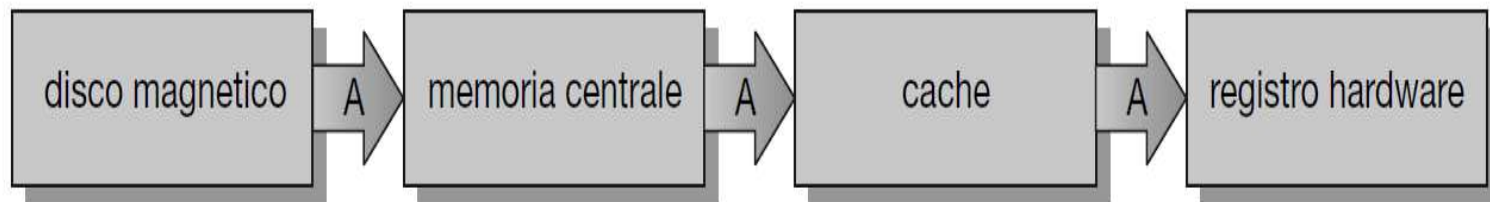
- Il SO gestisce:
  - Creazione e cancellazione file e directory
  - Primitive per modifica di file e directory
  - Mapping di file nella memoria secondaria
  - Backup di file su memoria non-volatile



# Introduzione

## Gestione cache

- Occorre utilizzare il dato più recente, indipendentemente da dove è archiviato nella gerarchia di archiviazione



**Figura 1.15** Migrazione di un intero A da un disco a un registro.

## Gestione I/O

- Il SO deve nascondere all'utente le specificità hardware dei dispositivi fornendo un'interfaccia uniforme

# Il Sistema Operativo Unix

**1965 Bell Laboratory** (con MIT e General Eletrics) lavora ad un nuovo sistema operativo: **Multics**. Principali caratteristiche: capacità multi-utente (multi-user), multi-processo (multi-processor) e un file system multi-livello (gerarchico) (multi-level file system).

**1969 AT&T abbandona Multics**. Ken Thompson, Dennis Ritchie, Rudd Canaday e Doug McIlroy, progettano e implementano la prima versione del file system Unix insieme ad alcune utility. Il nome Unix è di Brian Kernighan: gioco di parole su Multics.

**1 Gennaio 1970 Inizio di Unix.**

**Nel 1973 Unix riscritto in C** (Dennis Ritchie)

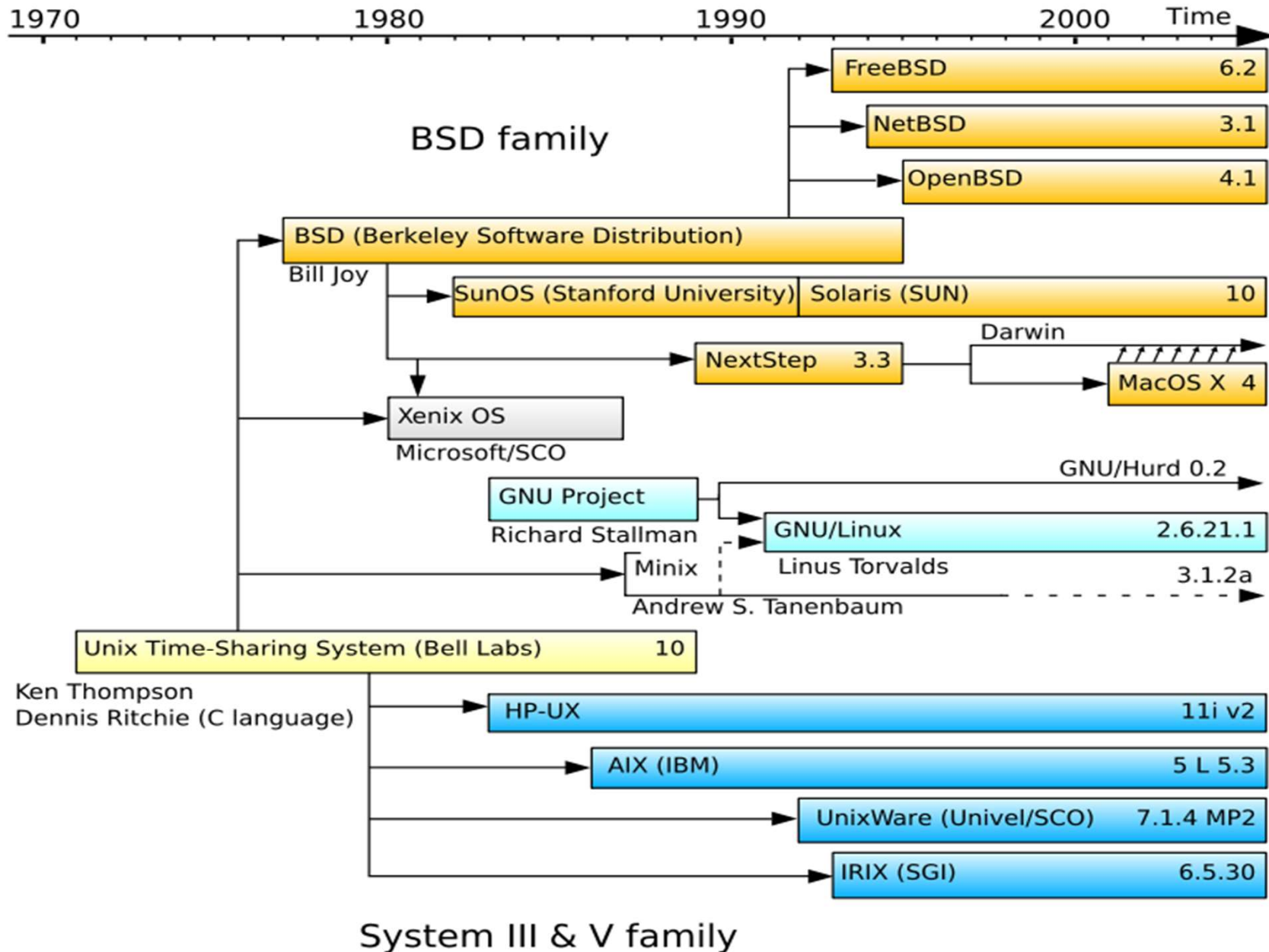
Caratteristiche:

- Architettura semplice ed elegante,
- Ambiente di programmazione (Implementato in C),
- Indipendenza dall'Hardware.

In particolare, SO basato su **kernel**

- il nucleo del SO e l'unica porzione che deve essere adattata all'Hw;
- tutte le altre funzioni poggiano sul nucleo (indipendenti Hw).

# Il Sistema Operativo Unix



# Architettura Sistema Unix

- Il **nucleo** del sistema (**kernel**) gestisce le risorse essenziali: la CPU, la memoria, le periferiche
- Tutto il resto, anche l'interazione con l'utente, è ottenuto tramite programmi eseguiti dal kernel, che accedono alle risorse hardware tramite delle richieste a quest'ultimo.

Il kernel è il solo programma ad essere eseguito in modalità privilegiata, con il completo accesso all'hardware, gli altri programmi vengono eseguiti in modalità protetta.

- Architettura **monolitica**

# Architettura di Sistema

Il kernel si occupa di:

**CPU:** Una parte del kernel, lo scheduler , si occupa di stabilire, ad intervalli fissi e sulla base priorità, quale “processo” deve essere mandato in esecuzione

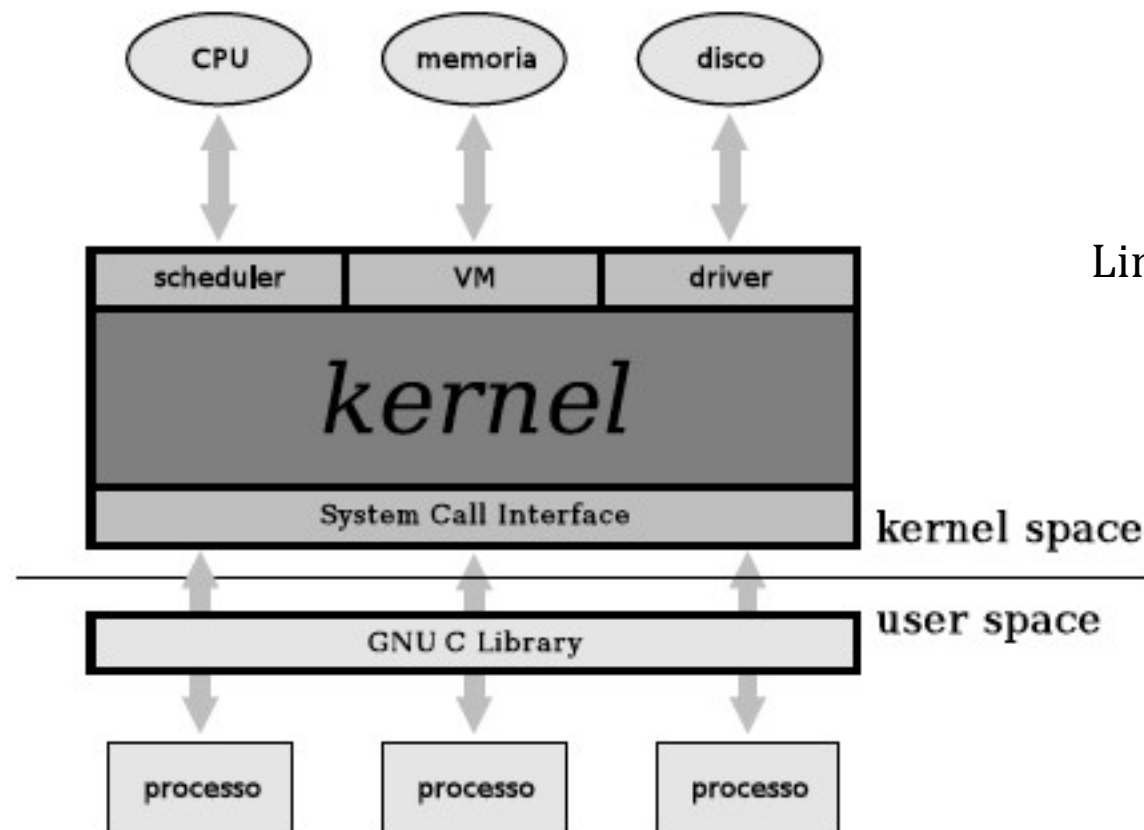
**Memoria:** La memoria viene gestita dal kernel attraverso il meccanismo della memoria logica, che consente di assegnare a ciascun processo uno spazio di indirizzi “virtuale” che il kernel, con l’ausilio della unità di gestione della memoria, rimappa automaticamente sulla memoria disponibile.

**Periferiche:** Le periferiche vengono viste attraverso un’interfaccia astratta che permette di trattarle come fossero file, secondo il concetto per cui “everything is a file” (le interfacce di rete fanno eccezione).



# Architettura di Sistema

Le interfacce con cui i programmi possono accedere all'hardware vanno sotto il nome di chiamate al sistema (**system call**): un insieme di funzioni che un programma può chiamare, per le quali viene generata un'interruzione del processo passando il controllo dal programma al kernel.



Linux architecture

Queste chiamate al sistema vengono rimappata in funzioni definite dentro opportune librerie (Libreria Standard del C).

# Sistema Multi-utente

- Il kernel Unix nasce fin dall'inizio come sistema multiutente
- Il concetto base è quello di utente (**user**) del sistema
- Sono previsti meccanismi per identificare gli utenti: ogni utente ha un nome (**username**) ed è identificato da uno user id (**uid**)
- Esistono **permessi** e **protezioni** per impedire che utenti diversi possano danneggiarsi a vicenda o danneggiare il sistema
- In ogni sistema è presente un utente speciale privilegiato, **superuser**, il cui username è di norma root, ed il cui uid è zero che identifica l'amministratore del sistema

# Unix Shell

- La shell è un interprete di comandi
- Si interpone tra l'utente ed il sistema operativo
- In sistemi Unix qualsiasi operazione può essere eseguita da una sequenza di comandi shell

Tra le Shell più utilizzate ci sono:

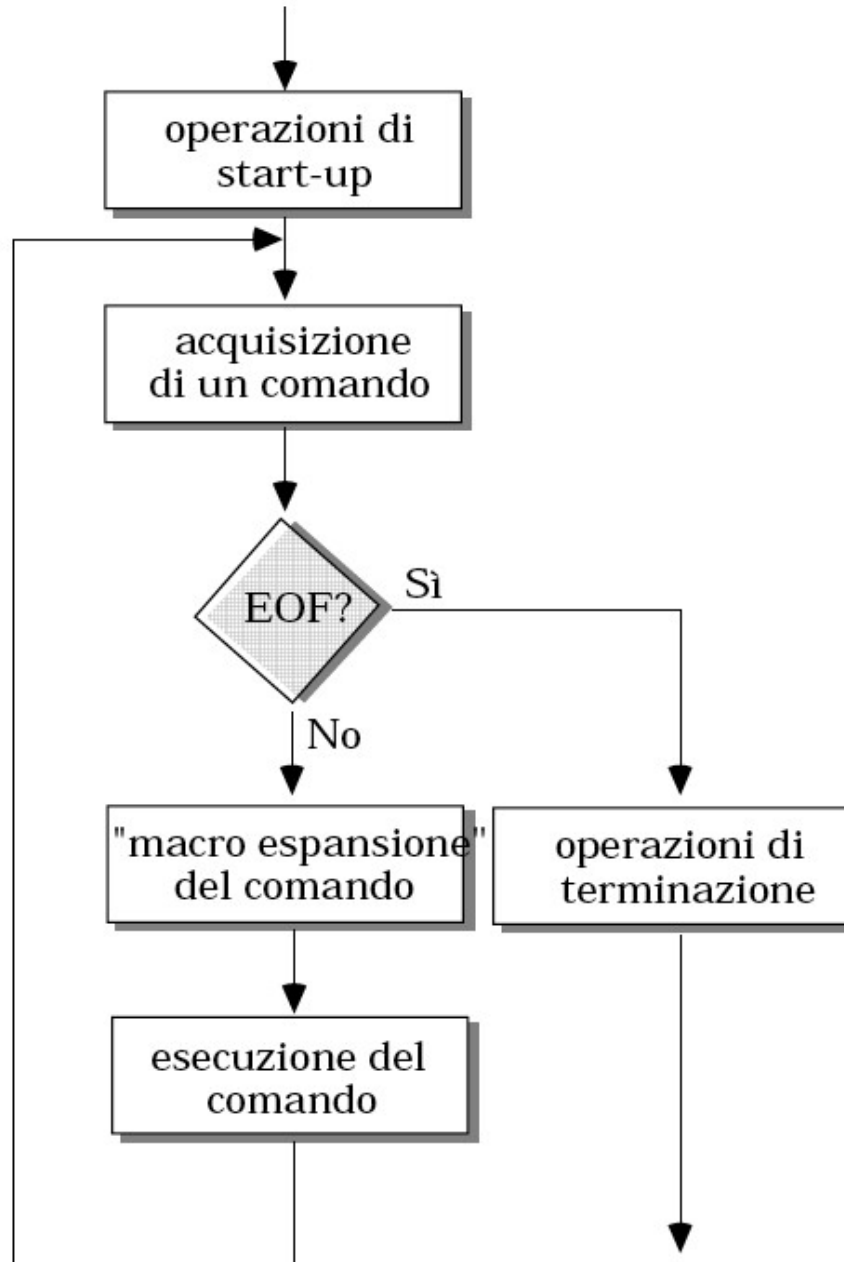
- |                       |           |             |
|-----------------------|-----------|-------------|
| • Bourne shell,       | /bin/sh   | (Bell Labs) |
| • Bourne-again shell, | /bin/bash | (Linux)     |
| • Cshell,             | /bin/csh  | (Berkeley)  |
| • Korn shell,         | /bin/ksh  | (Bell Labs) |
| • TENEX C shell       | /bin/tcsh | (BBN tech)  |

Il file `/etc/shells` contiene l'elenco delle shell installate dall'amministratore e disponibili a tutti gli utenti.

# Unix Shell

- Programma che interpreta il linguaggio a linea di comando attraverso il quale l'utente utilizza le risorse del sistema.
- Permette la gestione di variabili e dispone di costrutti per il controllo del flusso delle operazioni.
- Generalmente eseguito in modalità interattiva, all'atto del login, restando attivo per tutta la durata della sessione di lavoro ed effettuando le seguenti operazioni:
  - Gestione del “main command loop”
  - Analisi sintattica
  - Esecuzione di comandi (“built-in”, file eseguibili) e programmi in linguaggio di shell (script)
  - Gestione dello standard I/O e dello standard error
  - Gestione dei processi da terminale

# Ciclo Esecuzione Shell



# Shell Interattiva

- Comunicazione tra utente e shell avviene tramite comandi o script:
- Nome comando built-in oppure
- Nome di un file eseguibile oppure
- Nome di Script, cioè file ASCII presente nel sistema dotato del premezzo di esecuzione

```
$ ls -l
totale 48
drwxr-xr-x  2 root root  4096 dic 28 12:42 backups
drwxr-xr-x 17 root root  4096 nov 24 14:13 cache
drwxrwxrwt  2 root root  4096 dic 31 11:01 crash
drwxr-xr-x 71 root root  4096 dic 27 19:37 lib
drwxr-sr-x  2 root staff  4096 apr 12  2016 local
lrwxrwxrwx  1 root root    9 dic 27 19:11 lock -> /run/lock
drwxr-xr-x 13 root syslog 4096 gen  7 14:50 log
drwxr-sr-x  2 root mail  4096 nov 24 13:36 mail
drwxr-xr-x  2 root root  4096 nov 24 13:36 opt
lrwxrwxrwx  1 root root    4 dic 27 19:11 run -> /run
drwxr-xr-x  7 root root  4096 nov 24 13:39 spool
drwxrwxrwt 21 root root  4096 gen  7 14:50 tmp
$
```

WWW.ANDREAWEBB.COM

# Formato dei comandi

**comando [ argomento ...]**

Gli argomenti possono essere:

- opzioni o flag (-)
- parametri

separati da almeno un separatore (di default il carattere spazio)

L'ordine delle opzioni e', in genere, irrilevante

L'ordine dei parametri e', in genere, rilevante

Attenzione: **Unix e' CASE SENSITIVE**

# Formato dei comandi

## Comandi Equivalenti:

ls -l -F file1 file2 file3

ls -lF file1 file2 file3

ls -F -l file1 file2 file3

## Comandi NON equivalenti:

cat file1 > file2

cat file2 > file1

ls -l -F file1

ls -f -L file1

## Comandi ERRATI:

LS -F -L

CAT file1 > file2



# Esempi di Comandi

| Comando                      | Significato                                                    |
|------------------------------|----------------------------------------------------------------|
| <b>ps</b>                    | visualizza la lista dei processi lanciati dal terminale (pid)  |
| <b>ls</b>                    | visualizza la lista di file nella directory                    |
| <b>cd directory</b>          | cambia directory                                               |
| <b>passwd</b>                | cambia password                                                |
| <b>file filename</b>         | visualizza il tipo di file o il tipo di file con nome filename |
| <b>cat textfile</b>          | rivera il contenuto di textfile sullo screen                   |
| <b>pwd</b>                   | visualizza la directory di lavoro corrente                     |
| <b>exit</b> or <b>logout</b> | lascia la sessione                                             |
| <b>man <i>command</i></b>    | leggi pagine manuale su <b>command</b>                         |
| <b>info <i>command</i></b>   | leggi pagine Info su <b>command</b>                            |
| ...                          |                                                                |

# Classi di Comandi

- reperire informazioni su comandi, programmi, file....
- gestire filesystem
- operare su file e directory
- elaborare testi
- sviluppare software
- operare in remoto
- ....(comandi “di utilità” vari)
- amministrare il sistema
  - utenti e gruppi
  - dispositivi
  - software

# Listing di Processi

**ps** = process status → mostra i processi attivi.

Senza opzioni, elenca solo i processi collegati al terminale corrente e all'utente attivo (effective user).

Permette di osservare:

- PID (Process ID) → identificativo univoco del processo
- TTY → identificativo del terminale
- TIME → indica il tempo (hh:mm:ss) cumulato di utilizzo CPU
- CMD → comando che ha generato il processo

**ps -f**

full information (comando Unix style)  
UID, PID, PPID, C, STIME, TTY, TIME, CMD  
(C sta per percentuale di utilizzo CPU)

**ps -e** (oppure -A)

info su tutti i processi indipendentemente dal terminale

**ps -ef**

info su tutti i processi con dettagli

**ps a**

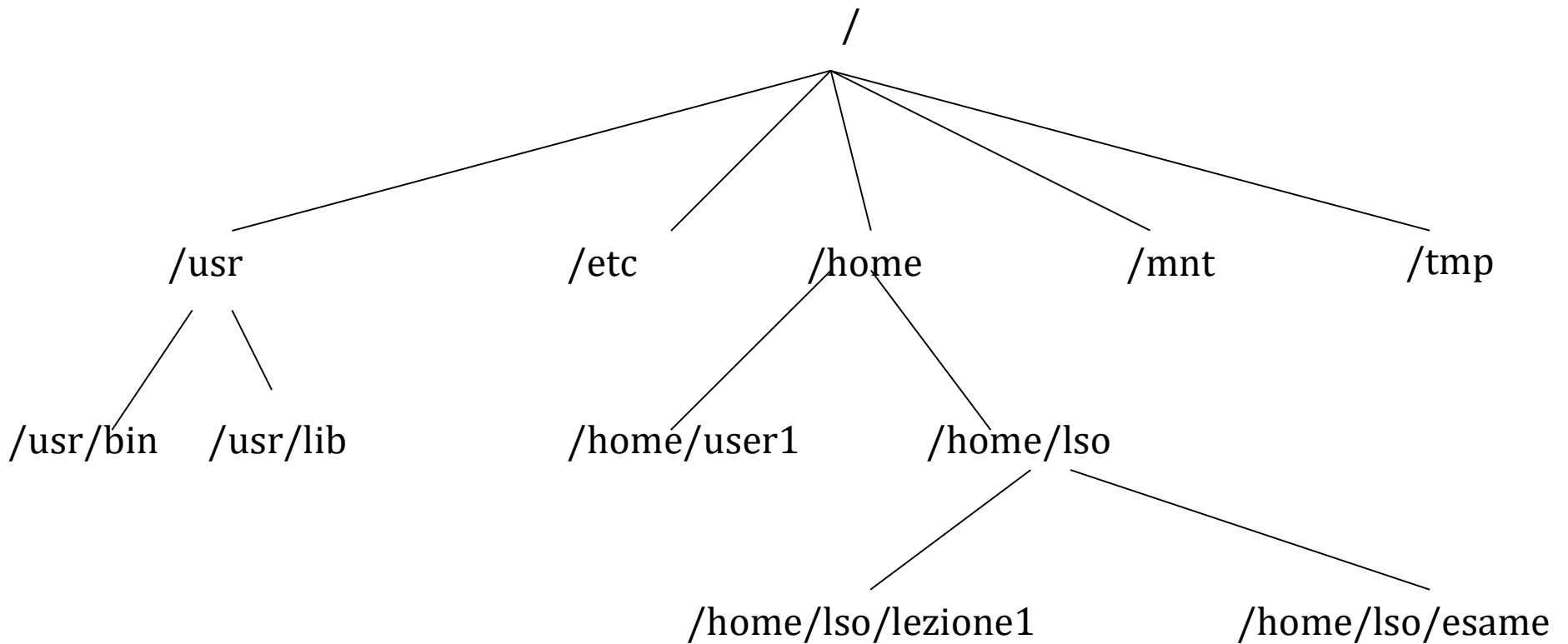
tutti i processi con terminale, non demoni (BSD style)

STAT indica lo stato (**R**unning, **S**leeping, **Z**ombi, etc.)

**ps aux**

x senza terminale, u in formato utente

# File System di UNIX



# Gestione delle Directory

`mkdir`

Crea una directory

`rmdir`

Elimina una directory vuota

`pwd`

Emette il percorso della directory corrente

`ls`

Elenca il contenuto di una o più directory

`du`

Calcola lo spazio utilizzato da una serie di directory e subdirectory

# mkdir (make directory)

`mkdir [options] directory`

Crea una directory secondo le opzioni

Alcune opzioni:

`-m mode` Indica le protezioni per la directory da creare

`-p` Crea, se assente, l'intero percorso indicato

Esempi:

```
mkdir -m 600 tracce_esami
```

```
mkdir -p lso/11-12/lezione1/
```

# rmmdir (remove directory)

`rmmdir [options] directory`

Rimuove la directory indicata.

Le directory possono essere rimosse se

(a) sono vuote e

(b) possibile scrivere nella directory padre.

Opzioni:            -p     Rimuove l'intero percorso indicato

Esempi:

Crea la directory lezione1 nella directory corrente

```
rmmdir lezione1
```

Rimuove, dalla directory corrente, il path indicato

```
rmmdir -p lso/11-12/lezione1/
```

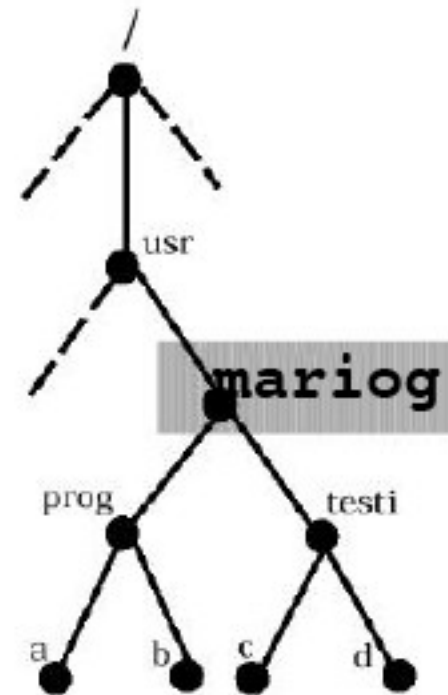
# pwd (print working directory)

**pwd** "printworking directory"

stampa il path della directory corrente

Esempio:

```
% pwd
/usr/mariog
%
```





# cd (change directory)

`cd [directory]`

Cambia la directory corrente a quella indicata.

Se non viene passato nessun argomento, la directory corrente diventa la home directory.

Esempio:

```
lso:~>cd
```

```
lso:~>pwd
```

```
/home/lso
```

```
lso:~>cd esempio/esempiocd
```

```
lso:~>pwd
```

```
/home/lso/esempio/esempiocd
```

# ls (list)

**ls [options] [directory]**

Elenca i file contenuti nella directory specificata.

Se la directory non e' indicata, viene elencato il contenuto della directory corrente.

Alcune opzioni:

- a Elenca anche i file nascosti
- l Formato esteso con informazioni su modo, proprietario, dimensione, etc dei file
- s fornisce la dimensione in blocchi dei file
- t lista i file nell'ordine di modifica (prima il file modificato per ultimo)
- 1 elenca i file in una singola colonna
- F aggiunge / al nome delle directory e \* al nome dei file eseguibili
- R si chiama ricorsivamente su ogni subdirectory

# ls - campi del formato esteso

Totale dimensione occupata (in blocchi)

|            | Riferimenti al file |     | Dimensione (byte) |      | Nome        |   |  |
|------------|---------------------|-----|-------------------|------|-------------|---|--|
| ls:~>ls -l |                     |     |                   |      |             |   |  |
| total 12   |                     |     |                   |      |             |   |  |
| -rw-rw-r-- | 1                   | lso | lso               | 10   | Mar 4 13:29 | a |  |
| -rw-rw-r-- | 1                   | lso | lso               | 10   | Mar 4 14:12 | b |  |
| drwxrwxr-x | 2                   | lso | lso               | 4096 | Mar 4 14:29 | c |  |

Tipo

Permessi

Proprietario Gruppo primario Data ultima modifica

(r)ead, (w)rite, e(x)ecute, (s)et uid bit

(d)irectory, (l)ink, (c)haracter special file, (b)lock special file, (-) ordinary file

# du (disk usage)

**du [options] file ...**

visualizza informazioni sull'utilizzo dello spazio (blocchi) da parte del/i file in esame. Il file puo' essere ordinario o un directory.

Se il file e' una directory, viene visualizzato, ricorsivamente, lo spazio utilizzato dalle sotto-directory

Alcune opzioni:

- s Visualizza lo spazio complessivamente occupato dal file in esame.
- k Visualizza lo spazio utilizzato espresso in Kb.

# Gestione dei File

|       |                                                                                                      |
|-------|------------------------------------------------------------------------------------------------------|
| cp    | Copia un file in un altro                                                                            |
| ln    | Crea un collegamento                                                                                 |
| mv    | Rinomina/sposta un file                                                                              |
| rm    | Cancella un file                                                                                     |
| chown | Cambia proprietario e gruppo primario di un file                                                     |
| chmod | Cambia i permessi di un file                                                                         |
| touch | Cambia il tempo di ultimo accesso ad un file                                                         |
| file  | Determina il tipo di file                                                                            |
| find  | Ricerca file in base ad opportuni criteri, permettendo di eseguirvi (opzionalmente) delle operazioni |
| tar   | archivia un file                                                                                     |
| gzip  | Comprime (Decomprime) un file                                                                        |

# cp (copy)

**cp [options] source... target**

copia un file in un altro file oppure uno o più file in una directory

Se vengono specificati solo i nomi di due file, il primo viene copiato sul secondo

Se vengono specificati solo due nomi, e se il secondo nome indicato è una directory, source viene copiato con lo stesso nome nella directory target. Se source è una directory, la copia avviene solo con opzioni particolari.

Se vengono indicati più di due nomi, il file target deve essere una directory e vengono generate le copie dei source in target. In mancanza di opzioni particolari, le directory non vengono copiate.

Alcune opzioni

-r se source e target sono directory, copia ricorsivamente source, i suoi file e le sue subdirectory in target

-i opera in modo interattivo, chiedendo una conferma se la copia comporta la cancellazione di un target preesistente

# Esempi

- Copia il file pippo nella directory corrente nel file /tmp/pippo.back

```
cp pippo /tmp/pippo.back
```

- Copia il file /tmp/pippo.back e la directory dir nella directory nuovadir

```
cp -r /tmp/pippo.back dir nuovadir
```

- Copia il file pippo nel file pippo2

```
cp pippo pippo2
```

# mv (move)

**mv [options] source... destination**

rinomina (sposta) file o directory.

Se vengono specificati solo i nomi di due elementi, source viene rinominato in destination, oppure in destination/source, a seconda che destination indichi un file o una directory. Qualora destination denoti un file preesistente, questo non sarà più accessibile come tale, e non sarà più accessibile in alcun modo se destination era il suo unico nome.

Se vengono indicati più di due elementi, destination deve essere una directory, e source\_1...source\_n vengono rinominati come destination/source\_1...destination/source\_n.

Nel caso che source e destination appartengono a due diversi file system, il comando effettua un vero e proprio spostamento dati tra i due file system. In tal caso vengono spostati solo i file ordinari, quindi: né collegamenti, né directory.

Alcune opzioni:

-i il comando chiede conferma all'utente qualora destination è un file preesistente



# rm (remove)

```
rm [options] file...
```

elimina i file o le directory indicati come argomento.

Alcune opzioni:

-i chiede conferma prima di rimuovere ogni file

-R rimuove ricorsivamente i file e le sottodirectory

Esempi d' uso

Elimina i file pippo nella directory corrente e /tmp/pippo.back

```
rm pippo /tmp/pippo.back
```

Elimina la directory nuovadir e tutto il suo contenuto

```
rm -R nuovadiru
```

Elimina tutti i file nella directory corrente

```
rm *
```

# touch

---

**touch [options] file...**

Cambia sia il tempo di ultimo accesso che di ultimo aggiornamento dei file. Per default, il tempo viene aggiornato alla data/ora di sistema al momento dell'invocazione del comando.

Se si specificano file che non esistono, questi vengono creati vuoti.

Alcune opzioni:

- a cambia solo il tempo di ultimo accesso
- c se file non esiste non viene creato
- m cambia solo il tempo di ultima modifica

```
% touch 01281738 file1
% ls -l
total 0
----r--r-- 1 mariog  staff 0 Jan 28 17:38 file1
```